

Lecture 26

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

October 5, 2026

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:
  $E.\textit{code}$

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:
  $E.\textit{code}$
  $\textit{if}\ E.\textit{place} == 0\ \textit{goto}\ S.\textit{after}$

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}:$
  $E.\textit{code}$
  $\textit{if}\ E.\textit{place} == 0\ \textit{goto}\ S.\textit{after}$
  $S_1.\textit{code}$

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:
  $E.\textit{code}$
  $\textit{if}\ E.\textit{place} == 0\ \textit{goto}\ S.\textit{after}$
  $S_1.\textit{code}$
  $\textit{goto}\ S.\textit{begin}$

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:
  $E.\textit{code}$
  $\textit{if}\ E.\textit{place} == 0\ \textit{goto}\ S.\textit{after}$
  $S_1.\textit{code}$
  $\textit{goto}\ S.\textit{begin}$
  $S.\textit{after}$:

Flow of Control

- $S \rightarrow \textit{while}\ E\ \textit{do}\ S_1$
- $S.\textit{begin}$:
  $E.\textit{code}$
  $\textit{if}\ E.\textit{place} == 0\ \textit{goto}\ S.\textit{after}$
  $S_1.\textit{code}$
  $\textit{goto}\ S.\textit{begin}$
  $S.\textit{after}$:
- $S.\textit{begin} = \textit{newlabel}()$
  $S.\textit{after} = \textit{newlabel}()$
  $S.\textit{code} = \textit{gen}(S.\textit{begin}:)||E.\textit{code}||\textit{gen}(\textit{if}\ E.\textit{place} == 0 \quad \textit{goto}$
  $S.\textit{after})||S1.\textit{code}||\textit{gen}(\textit{goto}\ S.\textit{begin})||\textit{gen}(S.\textit{after}:)$

Flow of Control

- $S \rightarrow \textit{if}\ E\ \textit{then}\ S_1\ \textit{else}\ S_2$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$
  $S_1.code$

Flow of Control

- $S \rightarrow \textit{if}\ E\ \textit{then}\ S_1\ \textit{else}\ S_2$
- $E.code$
  $\textit{if}\ E.place == 0\ \textit{goto}\ S.else$
  $S_1.code$
  $\textit{goto}\ S.after$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$
  $S_1.code$
  $goto\ S.after$
  $S.else:$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$
  $S_1.code$
  $goto\ S.after$
  $S.else : S_2.code$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$
  $S_1.code$
  $goto\ S.after$
  $S.else : S_2.code$
  $S.after :$

Flow of Control

- $S \rightarrow if\ E\ then\ S_1\ else\ S_2$
- $E.code$
  $if\ E.place == 0\ goto\ S.else$
  $S_1.code$
  $goto\ S.after$
  $S.else : S_2.code$
  $S.after :$
- $S.else = newlabel()\quad S.after = newlabel()$
  $S.code = E.code||$
  $gen(if\ E.place == 0$ goto S.else$)\ ||$
  $S_1.code||\ gen(goto\ S.after)||$
  $gen(S.else :)||$
  $S_2.code||$
  $gen(S.after :)$



Names in Symbol Table

- $S \rightarrow id = E$

Names in Symbol Table

- $S \rightarrow id = E$
  p = lookup(id.place);
  if p ! = null then emit(p = E.place)
  else error

Names in Symbol Table

- $S \rightarrow id = E$
  p = lookup(id.place);
  if p ! = null then emit(p = E.place)
  else error
- $E \rightarrow id$

Names in Symbol Table

- $S \rightarrow id = E$
  p = lookup(id.place);
  if p ! = null then emit(p = E.place)
  else error
- $E \rightarrow id$
  p = lookup(id.name);
  if p ! = null then E.place = p
  else error

Addressing Array Elements

- Arrays are stored in a block of consecutive locations

Addressing Array Elements

- Arrays are stored in a block of consecutive locations
- Assume width of each element is w

Addressing Array Elements

- Arrays are stored in a block of consecutive locations
- Assume width of each element is w
- i^{th} element of array A begins in location

Addressing Array Elements

- Arrays are stored in a block of consecutive locations
- Assume width of each element is w
- i^{th} element of array A begins in location

$$base + (i - low) \times w \leftrightarrow i \times w + const$$

  where base is relative address of $A[low]$
- For $n - d$ arrays storage can be either row major or column major.

Addressing Array Elements

- Arrays are stored in a block of consecutive locations
- Assume width of each element is w
- i^{th} element of array A begins in location

$$base + (i - low) \times w \leftrightarrow i \times w + const$$

where base is relative address of $A[low]$

- For $n - d$ arrays storage can be either row major or column major.
- in case of 2-D array stored in row major form address of $A[i_1, i_2]$ can be calculated as

$$base + ((i_1 - low_1) x n_2 + i_2 - low_2) xw \leftrightarrow ((i_1 x n_2) + i_2) xw + constant$$

$n_2 = high_2 - low_2 + 1$

Type conversion within assignments

- $E \rightarrow E_1 + E_2$

Type conversion within assignments

- $E \rightarrow E_1 + E_2$
  $E.place = newtmp();$
  $if\ E_1.type = integer$ and $E_2.type = integer$
  $then\ emit(E.place\text{“}=\text{”}E_1.place \quad \text{“}int+\text{”} \quad E_2.place);$
  $E.type = integer;$

Type conversion within assignments

- $E \rightarrow E_1 + E_2$
  $E.place = newtmp();$
  if $E_1.type = integer$ and $E_2.type = integer$
  then $emit(E.place \text{“} = \text{”} E_1.place \quad \text{“}int + \text{”} \quad E_2.place);$
  $E.type = integer;$
- Similar code if both $E_1.type$ and $E_2.type$ are *real*

Type conversion within assignments

- $E \rightarrow E_1 + E_2$
  $E.place = newtmp();$
  $if\ E_1.type = integer$ and $E_2.type = integer$
  $then\ emit(E.place\text{“}=\text{”}E_1.place \quad \text{“}int+\text{”} \quad E_2.place);$
  $E.type = integer;$
- Similar code if both $E_1.type$ and $E_2.type$ are *real*
- else if $E_1.type = int$ and $E_2.type = real$

Type conversion within assignments

- $E \rightarrow E_1 + E_2$
  $E.place = newtmp();$
  $if\ E_1.type = integer$ and $E_2.type = integer$
  $then\ emit(E.place\text{“}=\text{”}\ E_1.place \quad \text{“}int+\text{”} \quad E_2.place);$
  $E.type = integer;$
- Similar code if both $E_1.type$ and $E_2.type$ are $real$
- else if $E_1.type = int$ and $E_2.type = real$then $u = newtmp();$
  $emit(u\text{“}=\text{”}\ inttoreal\ E_1.place);$
  $emit(E.place\text{“}=\text{”}\ u\text{“}real+\text{”}\ E_2.place);$
  $E.type = real;$
- similar code if $E_1.type$ is real and $E_2.type$ is integer

Boolean Expression

- Compute logical values

Boolean Expression

- Compute logical values
- Change the flow of control

Boolean Expression

- Compute logical values
- Change the flow of control
- Boolean operators are: `and or not`

Boolean Expression

- Compute logical values
- Change the flow of control
- Boolean operators are: `and or not`
- $E \rightarrow E$ or E
  | E and E
  | not E
  |(E)
  | id relop *id*
  | *true*
  | *false*

Methods of translation

- Evaluate similar to arithmetic expressions

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use `1` for `true` and `0` for `false`

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use `1` for `true` and `0` for `false`
  - `a or b and not c`

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use `1` for `true` and `0` for `false`
  - `a or b and not c`
  - t_1 = `not c`

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use 1 for `true` and 0 for `false`
  - `a or b and not c`
  - $t_1 =$ `not c`
  - $t_2 =$ `b and` t_1

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use 1 for `true` and 0 for `false`
  - `a or b and not c`
  - $t_1 =$ `not c`
  - $t_2 =$ `b and` t_1
  - $t_3 =$ `a or` t_2
- Implement by flow of control

Methods of translation

- Evaluate similar to arithmetic expressions
  - Normally use `1` for `true` and `0` for `false`
  - `a or b and not c`
  - $t_1 =$ `not c`
  - $t_2 =$ `b and` t_1
  - $t_3 =$ `a or` t_2
- Implement by flow of control
  - given expression E_1 `or` E_2
    if E_1 evaluates to *true*
    then E_1 or E_2 evaluates to *true*
    without evaluating E_2

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$
- $E \rightarrow E_1$ and E_2

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$
- $E \rightarrow E_1$ and E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}and\text{”} E_2.place)$

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$
- $E \rightarrow E_1$ and E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}and\text{”} E_2.place)$
- $E \rightarrow not\ E_1$

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$
- $E \rightarrow E_1$ and E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}and\text{”} E_2.place)$
- $E \rightarrow not\ E_1$
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} \quad \text{“}not\text{”} E_1.place)$

Syntax directed translation of boolean expressions

- $E \rightarrow E_1$ or E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}or\text{”} E_2.place)$
- $E \rightarrow E_1$ and E_2
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} E_1.place \text{“}and\text{”} E_2.place)$
- $E \rightarrow not\ E_1$
  $E.place = newtmp()$
  $emit(E.place \text{“} = \text{”} \quad \text{“}not\text{”} E_1.place)$
- $E \rightarrow (E1) \quad E.place = E_1.place$